POST-TRAINING LLMs

A Visual Guide for Beginners

How we take a raw language model and turn it into one that follows instructions, reasons, and behaves.

01 Pre-training vs Post-training

02 The Three Pillars

03 Fine-Tuning

04 Reinforcement Learning

05 Reward Modeling

06 Policy Optimization

07 Test-Time Scaling

08 DeepSeek R1 Case Study

09 Benchmarks & Evaluation

10 Open Challenges

Based on arXiv:2502.21321

10 pages · beginner level

SECTION 01

Pre-training vs Post-training

Think of training a language model like training a new employee. Pre-training is university — years of reading everything. Post-training is the first job — learning how to actually behave, follow instructions, and do the work well.

The two stages

Stage	What it means
Pre-training	Read the entire internet. Learn language patterns, facts, grammar. Cost: millions of dollars, months of compute.
Post-training	Teach the model to follow instructions, reason step-by-step, be helpful and safe. Cost: much less, much faster.

Why we need post-training

A purely pre-trained model is like an autocomplete on steroids — it can finish sentences, but it does not know what you want. Ask it “What is 2+2?” and it might reply “What is 2+3?” because that is a common pattern in its training data. It has no concept of “answer the question”.

Post-training fixes this in three ways:

- **Alignment** — make the model want to do what the user asks, not just predict the next word.
- **Reasoning** — teach it to think step-by-step instead of jumping to a final answer.
- **Safety** — make it refuse harmful requests and behave ethically.

The three pillars of post-training

PILLAR 1

Fine-tuning

Show it examples of good behavior. Supervised learning on curated data.

PILLAR 2

Reinforcement Learning

Let it try things, reward good answers, penalize bad ones.

PILLAR 3

Test-Time Scaling

Think harder at inference time — more compute, better answers.

The big picture. Pre-training builds the foundation. Post-training turns that foundation into a useful assistant. The paper argues these are interconnected — you almost always want all three pillars working together.

SECTION 02

The Three Pillars in Detail

The paper organizes all post-training methods into three categories. Learn these and you understand the entire field.

Pillar 1 — Fine-Tuning

Fine-tuning means taking a pre-trained model and continuing to train it on a smaller, curated dataset. The data is typically (instruction, response) pairs where humans wrote the ideal answer.

Type	Example
Instruction FT	Model learns to follow single-turn instructions.
Dialogue FT	Multi-turn conversations that maintain context.
CoT FT	Show the model step-by-step reasoning traces.
Domain FT	Specialized training for medical, legal, financial text.
Distillation	A big teacher model labels data for a smaller student.
Preference FT	Teach the model which of two answers is better.

Pillar 2 — Reinforcement Learning

RL teaches by reward, not by example. The model generates answers, a reward model scores them, and the model learns to produce higher-scoring answers. This is how models like ChatGPT learn to be helpful without anyone writing every ideal answer by hand.

Pillar 3 — Test-Time Scaling

Instead of training the model differently, you let it think longer at inference. Generate multiple answers, search through possibilities, refine iteratively — all without changing the model weights.

WHEN TO USE

Fine-Tuning

When you have high-quality examples and want the model to imitate a specific style or capability.

WHEN TO USE

Reinforcement Learning

When you know good vs bad outcomes but not the exact correct answer. Great for reasoning.

WHEN TO USE

Test-Time Scaling

When the model is already good but you want it to think harder on hard problems.

In practice, they combine. The DeepSeek-R1 pipeline (page 9) uses all three: SFT for formatting, RL for reasoning, and test-time scaling for hard problems. Fine-tuning gets you 80% of the way; RL and TTS push it further.

SECTION 03

Fine-Tuning — Teaching by Example

Fine-tuning is the simplest form of post-training. Show the model enough good examples, and it learns to imitate them. This is supervised learning with a language model.

How it works

#	Step	What happens
1	Collect data	Human experts write (prompt, ideal response) pairs.
2	Train	Standard next-token prediction on this curated dataset.
3	Evaluate	Test on held-out prompts. Check if responses match expectations.
4	Deploy	The fine-tuned model is now specialized.

The six flavors of fine-tuning

Type	Description
Instruction FT	Model learns to follow single-turn instructions.
Dialogue FT	Model learns multi-turn conversation with context.
CoT FT	Model learns to show its work, step by step.
Domain FT	Model specializes for a field (BioGPT for medicine, FinBERT for finance).
Distillation	A large teacher model generates training data for a small student.
Preference FT	Model learns which of two responses humans prefer.

The efficiency problem (and solution)

Fine-tuning a 70B-parameter model normally requires hundreds of GPUs and weeks of compute. Too expensive for most teams. The solution is **Parameter-Efficient Fine-Tuning (PEFT)** — methods that only train a tiny fraction of the model weights.

Method	What it does
LoRA	Insert small trainable adapter matrices alongside the frozen model weights.
QLoRA	Combine LoRA with 4-bit quantization — fine-tune a 65B model on a single consumer GPU.
Prefix Tuning	Learn a small set of prompt embeddings, leave the model untouched.
Adapters	Insert tiny trainable layers between the frozen transformer blocks.

> **The trade-off.** Fine-tuning makes the model better at the target task but risks *catastrophic forgetting* — it may lose general capabilities learned during pre-training. The paper calls this a key open challenge (page 10).

SECTION 04

Reinforcement Learning — Teaching by Reward

RL is fundamentally different from fine-tuning. Instead of showing the model the correct answer, you let it generate answers and then reward or penalize them. The model learns what “good” looks like through trial and error.

Why RL is hard for language models

Traditional RL works on games, robots, and control systems. Language models are harder because:

- **Vast action space** — the model picks one token from ~50,000 options at every step.
- **Delayed rewards** — whether a full response is good or bad may only become clear at the end.
- **Subjective rewards** — “good answer” is not a clean numerical score like “win the game”.
- **Multiple objectives** — helpfulness, safety, accuracy, and style all pull in different directions.

The three-step RL pipeline

#	Step	What happens
1	Supervised Fine-Tuning (SFT)	Start with a pre-trained model. Fine-tune on high-quality examples so it can at least follow the format.
2	Train a Reward Model	Collect human preferences ("which of these two answers is better?"). Train a model to predict human preference scores.
3	RL Fine-Tuning	Use the reward model to score the model's outputs. Update the model to maximize score.

The reward hacking problem

If the reward model is imperfect (and it always is), the model will find ways to game it. It might learn that longer answers score higher, so it pads every answer. Or that certain phrases trigger the reward model, so it repeats them.

The standard fix is a **KL divergence penalty** — a term that penalizes the model for drifting too far from its original behavior. It is a leash that keeps the model from wandering into reward-hacking territory.

> **The key insight.** RL is how you teach a model things that are hard to specify explicitly — like “be helpful” or “reason step by step”. You cannot write down the perfect answer, but you can recognize a good one when you see it.

SECTION 05

Reward Modeling — Teaching a Model to Judge

Before you can do RL, you need a reward signal. A reward model is a second language model trained to score how good a response is. It learns from human preferences: “Given these two answers, which is better?”

How a reward model is trained

#	Step	What happens
1	Collect preference data	For many prompts, humans rank or compare the model's answers.
2	Train the reward model	Given (prompt, response), output a scalar score. Learn to match human rankings.
3	Use it during RL	The main policy model generates answers. The reward model scores them. The policy learns to maximize score.

Two types of reward models

Type	How it works
Outcome Reward Model (ORM)	Scores only the final answer. Simple, but cannot tell *why* an answer was good.
Process Reward Model (PRM)	Scores every intermediate reasoning step. More accurate, but requires much more annotation.

The three sources of reward signal

HUMAN FEEDBACK

RLHF

Humans rank outputs. Highest quality but expensive and slow. Used by ChatGPT.

AI FEEDBACK

RLAIF

A strong AI model judges outputs instead of humans. Cheaper, scalable, close to RLHF quality.

RULE-BASED

Verifiable Rewards

For math or code, correctness is directly checkable. No human or AI judge needed.

The hardest problem. Reward models are imperfect proxies for what humans actually want. The paper calls this “reward hacking” — the model optimizes the proxy without achieving the real goal. Robust reward modeling is one of the biggest open problems in the field (page 10).

SECTION 06

Policy Optimization — The Algorithms

Once you have a reward model, you need an algorithm that uses it to update the language model (the “policy”). The paper covers several. These names appear everywhere, so learn what each one does at a high level.

The algorithm family

Algorithm	What makes it special
PPO (2022)	The classic. Uses a value model as a critic. Clips updates to stay stable. Powerful but complex. Used in original RLHF.
DPO (2023)	Direct Preference Optimization. Skips the reward model entirely — learns directly from pairwise preferences. Simpler, cheaper.
GRPO (2024)	Group Relative Policy Optimization. Removes the critic by comparing outputs within a group. Powers DeepSeek-R1. Very efficient.
ORPO (2024)	Odds Ratio Preference Optimization. Combines SFT and preference learning in one step. Simplest of all.
TRPO (2015)	Trust Region Policy Optimization. PPO's predecessor. Theoretically clean, computationally expensive. Rarely used today.

PPO vs DPO vs GRPO — the intuition

Algorithm	Plain-English version
PPO	"Try things, get scored, learn from the score, but don't change too fast." Needs a value function to estimate future reward.
DPO	"Here are two answers, this one is better. Learn to prefer it." No reward model, no RL loop. Just contrastive learning.
GRPO	"Generate 8 answers, see which are better than average, learn from the winners." No value function needed. Cheap and effective.

Why GRPO matters

GRPO is the key innovation behind DeepSeek-R1 (page 9). It removes the need for a separate value network, which roughly halves memory usage and makes RL training much cheaper. Instead of asking “how good is this answer compared to my value estimate?”, it asks “how good is this answer compared to the other answers I generated for the same prompt?” That simple change enables RL on a much larger scale.

> **What they share.** Every algorithm uses the same core idea: generate responses, score them, update the model to increase the probability of high-scoring responses. They differ in *how* they estimate “high-scoring” and *how* aggressively they update.

SECTION 07

Test-Time Scaling — Thinking Harder

So far, everything has been about changing the model. Test-time scaling (TTS) is different — it changes how the model is *used* at inference time, without touching the weights. Give it more compute, get better answers.

The big idea

A small model with lots of thinking time can beat a much larger model with no thinking time. The paper reports that a smaller model with TTS can match a 14x larger model on easy-to-medium tasks. This is huge — it means you do not always need to train bigger models.

The main TTS techniques

Technique	What it does
Chain-of-Thought (CoT)	Prompt the model to "think step by step" before answering.
Self-Consistency	Generate many reasoning chains, take the majority vote for the final answer.
Best-of-N	Generate N answers, use a reward model to pick the best one.
Beam Search	Keep the top-N partial sequences at each step, expand only the most promising.
Tree-of-Thought (ToT)	Branch out into multiple reasoning paths, backtrack when one fails.
Graph-of-Thought (GoT)	Like ToT but thoughts can merge and refine, not just branch.
Monte Carlo Tree Search	Simulate many rollouts, use statistics to focus on promising paths.
Self-Refinement	Generate, critique, refine, repeat until good enough.

The cost trade-off

TTS is not free. Each of these techniques multiplies the compute used at inference. If you generate 20 answers instead of 1, you spend 20x the compute. The paper introduces **Compute-Optimal Scaling (COS)** to solve this — estimate each prompt's difficulty, then allocate compute accordingly. Easy questions get 1 answer, hard ones get 20. This achieves 4x efficiency over uniform Best-of-N.

The 14x result. On easy to intermediate tasks (like MATH benchmarks), a small model with TTS can outperform a model 14x larger. This is the strongest argument for test-time scaling — it is often cheaper than training a bigger model.

SECTION 08

Case Study — DeepSeek-R1

DeepSeek-R1 is the paper's flagship example. It was trained almost entirely with RL (not SFT) and matches or beats OpenAI's o1 on reasoning benchmarks. Here is how they did it, step by step.

The DeepSeek-R1 training pipeline

#	Step	What happens
1	Cold-Start SFT	Fine-tune the base model on a small set of curated reasoning examples. This teaches the format: <reasoning>...</reasoning><answer>...</answer>.
2	RL for Reasoning	Use GRPO to train the model on math, code, and logic problems where correctness is directly checkable. The model learns to think step by step.
3	Rejection Sampling	The RL-trained model generates hundreds of thousands of correct solutions. Filter for high quality. Mix with existing SFT data.
4	Second SFT	Fine-tune again on this expanded dataset. This is now called the "reasoning-oriented SFT" phase.
5	Second RL	A second RL round on the new SFT model. This time the reward includes not just correctness but also helpfulness and formatting.
6	Distillation	The final R1 model acts as a teacher for smaller models (Qwen, Llama families, 1.5B to 70B parameters).

Why this works

The key insight is that pure RL (no SFT) is unstable — the model generates garbage at first, gets random rewards, and never converges. The cold-start SFT teaches the model what a good answer looks like, then RL amplifies that capability. The paper calls this "cold-start reinforcement learning".

> **The remarkable part.** R1-Zero (pure RL, no cold-start SFT) still learned to reason, but it was unreadable — it mixed languages, produced garbled output, and wandered. The cold-start data fixed this by teaching format first, then reasoning. This is why the paper argues SFT and RL are complementary, not competing.

SECTION 09

Benchmarks & Open Challenges

How do we know if post-training actually helped? Benchmarks.

Benchmark	What it tests
MATH / GSM8K	Step-by-step math problems. The standard test for reasoning.
MMLU	57-subject multiple-choice test. The standard test for knowledge.
TruthfulQA	Does the model make things up? Tests factual accuracy.
HumanEval / CodeT5	Can the model write correct code?
MT-Bench / Chatbot Arena	Multi-turn conversation quality, judged by humans.
UltraFeedback / HelpSteer	Tests preference alignment — does the model produce what users prefer?
TydiQA / XGLUE	Multilingual capability.

The paper's five biggest open problems

- **Catastrophic forgetting.** Fine-tuning on new data can erase old capabilities. LoRA helps but does not fully solve it.
- **Reward hacking.** The model games the reward model instead of learning the real goal. Robust reward modeling is unsolved.
- **The overthinking problem.** Models sometimes generate 22% wasted computation on reasoning chains longer than needed.
- **Efficient RL.** RL training is still 10-100x more expensive than fine-tuning. Cheaper methods are needed.
- **Privacy.** Personalized models risk leaking private training data through memorization.

The future direction

The paper argues that pre-training and post-training are not competing — they are complementary. Future systems will likely be hybrids: smaller pre-trained base models, with heavy post-training for reasoning and test-time scaling for hard problems. The 14x scaling result means you might not need to train a bigger model at all — just a smarter training pipeline and more inference-time compute.

The one thing to remember. Pre-training teaches a model language. Post-training teaches it to *use* language — to follow instructions, reason, be helpful, be safe. In 2025 and beyond, the frontier is not in bigger models, it is in smarter post-training.

LLM POST-TRAINING · BEGINNER GUIDE

Train the foundation, then teach it to think.

10 pages · beginner level · based on arXiv:2502.21321